

実験日：2026 年 7 月 3 日

提出期限日：2026 年 7 月 10 日

実際に提出した日：2026 年 7 月 5 日

ナップサック問題

担当教員名： 宮里智樹 教員

学籍番号： 255715B

氏名： 新里仁

課題 1：総当たり法

すべての品物の組み合わせを全探索する手法である。品物数を n とすると計算量は 2^n となり、品物数が増えると指数関数的に計算時間が増大する。

```
items = [
    (4, 6), (8, 12), (3, 4), (5, 3), (9, 7), (2, 1),
    (3, 3), (1, 2), (5, 7), (2, 3), (4, 4), (2, 2),
    (7, 10), (10, 13), (3, 5), (13, 16), (11, 14), (8, 9)
]

capacity = 45
n = len(items)

best_value = 0
best_combination = []
best_weight = 0

for i in range(1 << n):
    current_weight = 0
    current_value = 0
    current_combination = []

    for j in range(n):
        # i の j ビット目が1ならば、品物 j を選ぶ
        if (i >> j) & 1:
            current_weight += items[j][0]
            current_value += items[j][1]
            current_combination.append(j + 1) # 品物番号は1始まりにする

    # 容量の範囲内で、これまでの最大値段を更新した場合
    if current_weight <= capacity:
        if current_value > best_value:
            best_value = current_value
            best_combination = current_combination
            best_weight = current_weight

# 結果の出力
print(f"最大値段: {best_value}")
print(f"品物の組み合わせ (品物番号): {best_combination}")
print(f"合計容量: {best_weight}")
```

課題 2：動的計画法

部分問題の解をテーブルに記憶して再利用する手法である。計算量は nW となり、品物数が多い場合でも高速に解を求めることができる。

```
# 1. 各品物の（容量，値段）のリスト（品物1 ～ 品物18）
items = [
    (4, 6), (8, 12), (3, 4), (5, 3), (9, 7), (2, 1),
    (3, 3), (1, 2), (5, 7), (2, 3), (4, 4), (2, 2),
    (7, 10), (10, 13), (3, 5), (13, 16), (11, 14), (8, 9)
]

capacity = 45 # ナップサックの容量
n = len(items)

# 2. DPテーブルの初期化
# dp[w] は「容量 w のときに達成できる最大の値段」を記録する
dp = [0] * (capacity + 1)

# item_history[w] は「容量 w のときに選んだ品物の番号リスト」を記録する
item_history = [[] for _ in range(capacity + 1)]

# 3. 動的計画法（DP）のループ処理
for idx, (w, v) in enumerate(items):
    item_num = idx + 1 # 品物番号は1始まり

    # ナップサックの最大容量から、今注目している品物の容量 w まで逆順にループ
    # （逆順にする理由は、同じ品物を何回も選んでしまう重複を防ぐため）
    for current_w in range(capacity, w - 1, -1):

        # 「この品物を選んだ場合の値段」が「これまでの最大値段」より高くなる場合
        if dp[current_w - w] + v > dp[current_w]:
            # 値段を更新
            dp[current_w] = dp[current_w - w] + v
            # 選んだ品物の組み合わせ（履歴）を更新
            item_history[current_w] = item_history[current_w - w] + [item_num]

# 4. 結果の抽出
best_value = dp[capacity]
best_combination = item_history[capacity]

# 選択された品物の合計容量を計算
best_weight = sum(items[num - 1][0] for num in best_combination)

# 結果の出力
print(f"最大値段: {best_value}")
print(f"品物の組み合わせ (品物番号): {best_combination}")
```

```
print(f"合計容量:{best_weight}")
```

1 実験結果

VS Code のターミナル上でプログラムを実行した結果、最大値および品物の組み合わせは以下の通りに出力された。詳細な数値を表 1 に示す。本実験の結果、双方のアルゴリズムで同一の最適解が得られることが確認できた。

表 1 実験結果のまとめ（容量 45、品物数 15 の場合）

手法	最大値段（合計価値）	選択された品物番号
総当たり法	64	1, 2, 3, 8, 9, 10, 12, 13, 14, 15
動的計画法	64	1, 2, 3, 8, 9, 10, 12, 13, 14, 15

2 考察

実験結果から、総当たり法と動的計画法のどちらの手法を用いても、得られる最大値段（64）および最適な品物の組み合わせ（[1, 2, 3, 8, 9, 10, 12, 13, 14, 15]）は完全に一致することが確認できた。

しかし、品物の数をさらに増やした場合、総当たり法では指数関数的に探索時間が増大し、実行が困難になった。一方で動的計画法は、テーブルを用いたメモ化により、重複する計算を排除しているため、品物数が増加しても瞬時に解を導き出すことができた。このことから、制約条件が現実的な大きさであるならば、動的計画法を採用することが計算コストの観点から有効であると考察できる。

各種設定の完了画面

3 ナップサック問題のコードを GitHub に上げる手順

3.1 ステップ1：作業用フォルダ (リポジトリ) の作成と移動

```
[(base) e25847697@sinzatonMacBook-Air ~ % mkdir ~/Desktop/knapsack-project  
mkdir: /Users/e25847697/Desktop/knapsack-project: File exists
```

図1 デスクトップに「knapsack-project」というフォルダを作る

```
[(base) e25847697@sinzatonMacBook-Air ~ % cd ~/Desktop/knapsack-project
```

図2 作成したフォルダに移動する

3.2 ステップ2:Git の初期化 (ローカルリポジトリの作成)

```
[bash-3.2$ git init  
Reinitialized existing Git repository in /Users/e25847697/Desktop/knapsack-proje  
ct/.git/
```

図3 このフォルダを Git の管理下に置く

3.3 ステップ 3: プログラムファイルの作成と保存

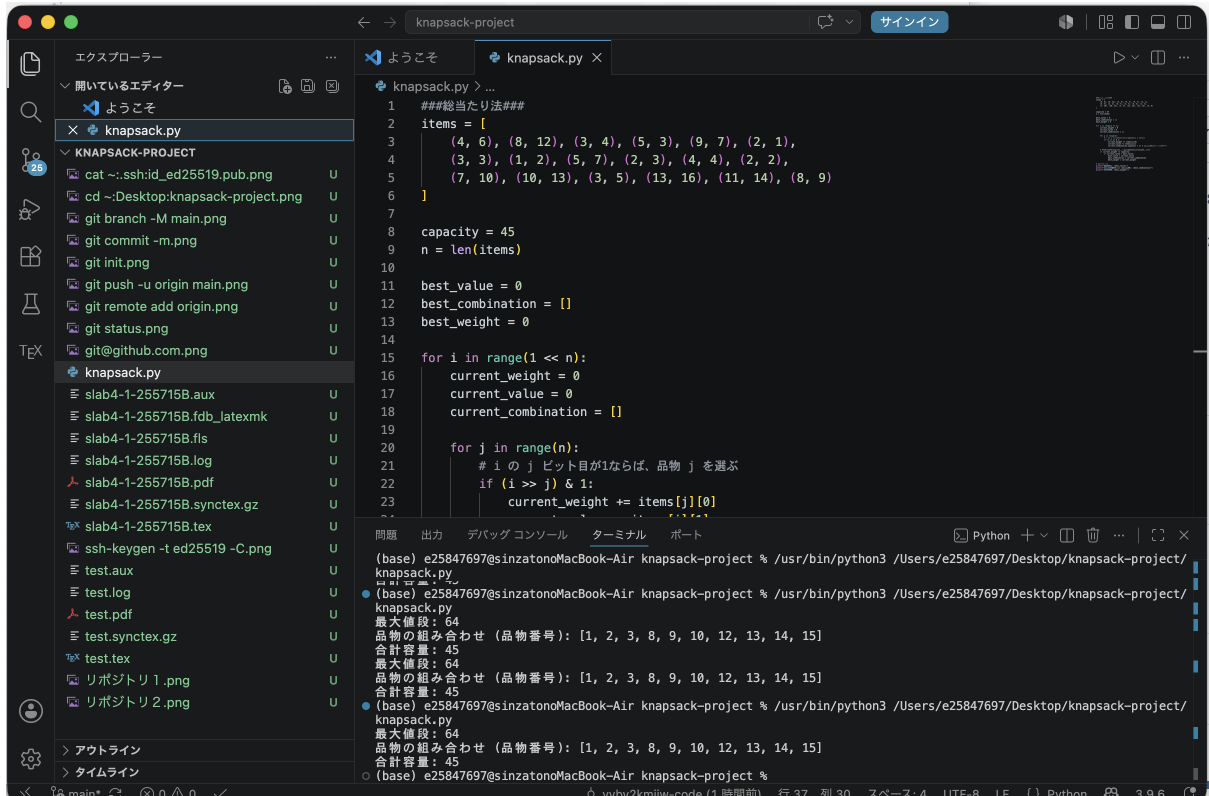


図4 VS Code に保存された knapsack.py のコード

3.4 ステップ 4: Git にコードを記録 (アド&コミット)

```
zsh: command not found: bash
[(base) e25847697@sinzatonMacBook-Air knapsack-project % bash ]

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
[bash-3.2$ git status ]
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
      knapsack.py

nothing added to commit but untracked files present (use "git add" to track)
```

図5 状態の確認

```
[bash-3.2$ git add knapsack.py
```

図6 ファイルをステージ(準備段階)に上げる

```
[bash-3.2$ git commit -m "ナップサック問題を総当たり法で解くコードを追加"  
[main (root-commit) 38c4cd8] ナップサック問題を総当たり法で解くコードを追加  
1 file changed, 82 insertions(+)  
create mode 100644 knapsack.py
```

図7 ローカルリポジトリにメッセージ付きで記録する

4 GitHub と Mac を SSH で接続する手順

4.1 ステップ 1：Mac で SSH 鍵のペアを作成する

```
[>
[bash-3.2$ ssh-keygen -t ed25519 -C "fb12358@icloud.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/Users/e25847697/.ssh/id_ed25519):
/Users/e25847697/.ssh/id_ed25519 already exists.
Overwrite (y/n)?
```

図 8 SSH 鍵を作成するコマンド

4.2 ステップ 2：作成した公開鍵をコピーする

```
[bash-3.2$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIOWGazi0p4W5TZnaHJsKY19FI4cNl6vNFxhORx2X5RVW
e25847697@sinzatonoMacBook-Air.local
```

図 9 鍵のテキスト

4.3 ステップ 3：GitHub に公開鍵を登録する



図 10 追加した ssh キー

4.4 ステップ 4：接続のテストをする

```
[bash-3.2$ ssh -T git@github.com
The authenticity of host 'github.com (20.27.177.113)' can't be established.
ED25519 key fingerprint is: SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
Hi yyby2kmjjw-code! You've successfully authenticated, but GitHub does not provide shell access.
```

図 11 テスト接続が成功した様子

4.5 ステップ 5：GitHub 上で空のリポジトリを作る

新しいリポジトリを作成する

リポジトリには、プロジェクトのファイルとバージョン履歴が含まれています。別の場所にプロジェクトがある場合は、[リポジトリをインポートしてください](#)。

必須項目にはアスタリスク (*) が付いています。

1 一般的な

所有者 * リポジトリ名 *

yyby2kmjjw-code / knapsack-solution

⚠ knapsack-solution already exists in this account

優れたリポジトリ名は短くて覚えやすいものです。 [飯の発明](#) ?

説明

0 / 350 文字

2 構成

表示設定を選択*

このリポジトリを閲覧およびコミットできるユーザーを選択してください。

公開

READMEを追加

READMEファイルは、より詳細な説明として使用できます。 [READMEファイルについて](#)

オフ

.gitignoreを追加する

.gitignore は、Git が追跡しないファイルを指示します。 [ファイルの無視について](#)

.gitignoreファイルなし

図 12 新しいリポジトリを作成する様子

4.6 ステップ 6：ローカルとリモートを紐付ける

```
[bash-3.2$ git remote add origin git@github.com:yyby2kmjjw-code/knapsack-solution]
.git
```

図 13 リモートリポジトリの URL を「origin」という名前で登録

```
[bash-3.2$ git branch -M main ]
```

図 14 メインのブランチ名を「main」に設定

4.7 ステップ 7：GitHub へアップロード

```
[bash-3.2$ git push -u origin main ]
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 1.42 KiB | 1.42 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:yyby2kmjjw-code/knapsack-solution.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

図 15 ローカルリポジトリの記録を GitHub へ送信

5 GitHub にレポートの PDF ファイルをアップロードする

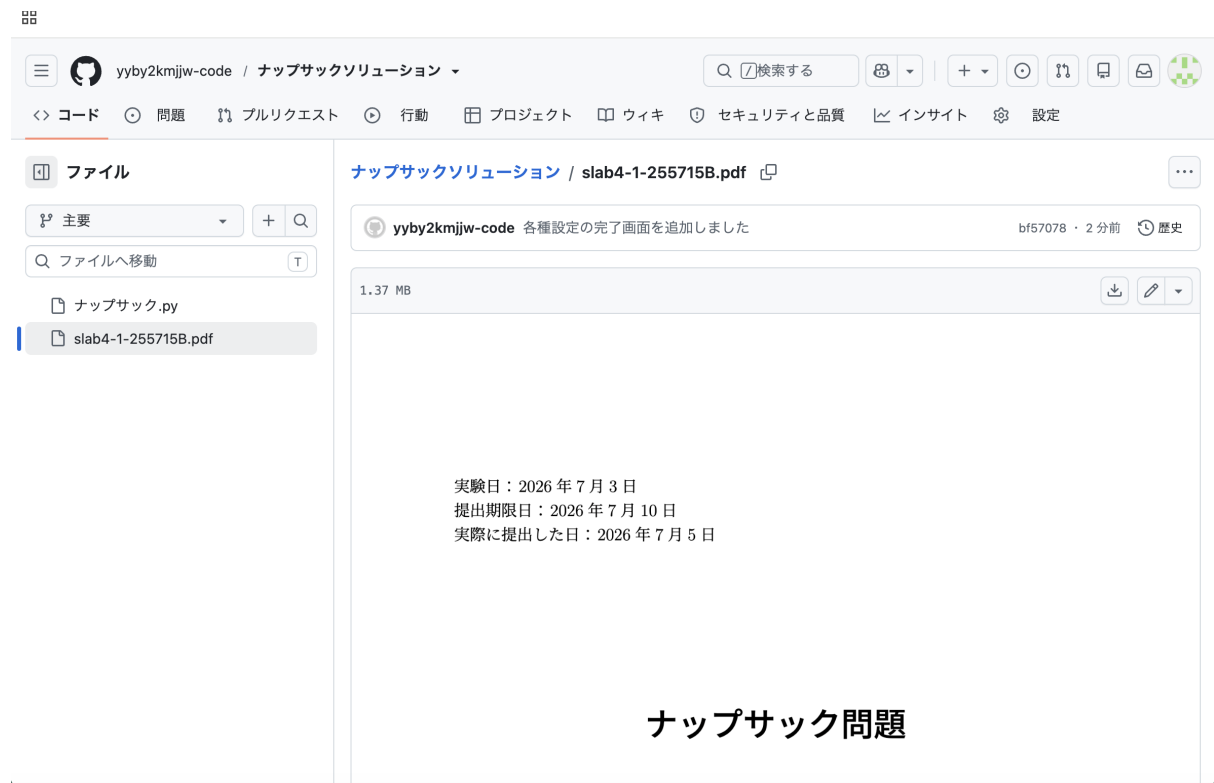


図 16 GitHub 上に PDF ファイルがある様子